

## ANSIBLE-TASK-3

### TASKS:

- ❖ Create ansible playbook to create ansible vault.
- ❖ Write a ansible playbook to install apache in linux and ubuntu machine by using when condition.
- ❖ Create ansible playbook using roles to configure LAMP stack
- ❖ Setup ansible AWX and explore the options.

### Task Overview & Importance (Ansible + AWX)

---

#### 1 Create Ansible Playbook to Create Ansible Vault

##### ◆ What was done

- Created an Ansible playbook that:
  - Prompts for vault password and secret
  - Creates a temporary password file
  - Encrypts a YAML file using ansible-vault
  - Cleans up sensitive files

##### ◆ Tools Used

- Ansible
- Ansible Vault

##### ◆ Importance

- Protects **sensitive data** such as passwords, keys, and tokens
- Prevents secrets from being stored in **plain text**
- Enables secure automation and CI/CD integration
- Essential for **security compliance** in production environments

### Why important?

Security is critical in DevOps. Ansible Vault ensures secrets are encrypted while still allowing automation.

---

## 2 Install Apache on Linux & Ubuntu Using when Condition

### ◆ What was done

- Created a single playbook to install Apache on:
  - **Ubuntu (apache2 via apt)**
  - **RHEL/Amazon Linux (httpd via yum)**
- Used when conditions based on OS type

### ◆ Tools Used

- **Ansible**
- **Ansible Facts**
- **apt / yum modules**

### ◆ Importance

- Eliminates the need for multiple playbooks per OS
- Makes automation **OS-aware and flexible**
- Reduces manual configuration errors
- Improves **reusability and scalability**

### Why important?

In real environments, infrastructure is heterogeneous. Conditional logic allows one playbook to work everywhere.

---

## 3 Configure LAMP Stack Using Ansible Roles

### ◆ What was done

- Created a **LAMP stack** using roles:
  - Apache role
  - MySQL (MariaDB) role
  - PHP role
- Used ansible-galaxy init for standard role structure

### ◆ Tools Used

- **Ansible Roles**
- **Ansible Galaxy**
- **YAML**
- ◆ **Importance**
  - Roles enable **modular, reusable, and maintainable code**
  - Easy to update or replace individual components
  - Follows **industry best practices**
  - Ideal for **team collaboration and scaling**

### **Why important?**

Roles are the foundation of enterprise Ansible automation and are required for large projects and AWX usage.

---

## **Setup Ansible AWX and Explore Options**

- ◆ **What was done**
  - Installed AWX using Docker
  - Configured:
    - Inventory
    - Hosts
    - Credentials
    - Job Templates
  - Executed Ansible playbooks from GUI
- ◆ **Tools Used**
  - **Ansible AWX**
  - **Docker**
  - **Docker Compose**
  - **Ansible**
- ◆ **Importance**
  - Provides a **GUI and centralized control** for Ansible

- Enables:
  - Role-Based Access Control (RBAC)
  - Job scheduling
  - Secure credential management
  - Real-time job monitoring
- Makes Ansible accessible to **non-CLI users**

## Why important?

AWX is used in enterprises to manage automation securely, at scale, with visibility and control.

## 1. Create ansible playbook to create ansible vault.

→ create a playbook create-vault.yml

```

root@ip-172-31-13-122:~/playbooks
- name: Create a full Ansible Vault file
  hosts: localhost
  gather_facts: no

  vars_prompt:
    - name: vault_password
      prompt: "Enter vault password"
      private: yes
    - name: secret_value
      prompt: "Enter the secret value"
      private: yes

  tasks:
    - name: Write vault password to a temporary file
      copy:
        content: "{{ vault_password }}"
        dest: /tmp/vault_pass
        mode: '0600'

    - name: Create plain YAML file with secret
      copy:
        content: |
          my_secret: "{{ secret_value }}"
        dest: /tmp/plain.yml

    - name: Encrypt the plain YAML file into a vault file
      command: >
        ansible-vault encrypt /tmp/plain.yml
        --vault-password-file /tmp/vault_pass

    - name: Rename encrypted file to vault.yml
      command: mv /tmp/plain.yml vault.yml

    - name: Remove temporary vault password file
      file:
        path: /tmp/vault_pass
        state: absent

```

→ Add this above code:

---

- name: Create a full Ansible Vault file

hosts: localhost

gather\_facts: no

vars\_prompt:

- name: vault\_password

prompt: "Enter vault password"

private: yes

- name: secret\_value

prompt: "Enter the secret value"

private: yes

tasks:

- name: Write vault password to a temporary file

copy:

content: "{{ vault\_password }}"

dest: /tmp/.vault\_pass

mode: '0600'

- name: Create plain YAML file with secret

copy:

content: |

my\_secret: "{{ secret\_value }}"

dest: /tmp/plain.yml

- name: Encrypt the plain YAML file into a vault file

command: >

ansible-vault encrypt /tmp/plain.yml

--vault-password-file /tmp/.vault\_pass

- name: Rename encrypted file to vault.yml

command: mv /tmp/plain.yml vault.yml

- name: Remove temporary vault password file

file:

path: /tmp/.vault\_pass

state: absent

→run playbook using code “ansible-playbook create-vault.yml

```
[root@ip-172-31-13-122 playbooks]# vi create-vault.yml
[root@ip-172-31-13-122 playbooks]# ansible-playbook create-vault.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'
Enter vault password:
Enter the secret value:

PLAY [Create a full Ansible Vault file] *****
TASK [Write vault password to a temporary file] *****
changed: [localhost]
TASK [Create plain YAML file with secret] *****
changed: [localhost]
TASK [Encrypt the plain YAML file into a vault file] *****
changed: [localhost]
TASK [Rename encrypted file to vault.yml] *****
changed: [localhost]
TASK [Remove temporary vault password file] *****
changed: [localhost]
PLAY RECAP *****
localhost                : ok=5  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

[root@ip-172-31-13-122 playbooks]#
```

## 2. Write a ansible playbook to install apache in linux and ubuntu machine by using when condition.

→apachi.yml

```
[root@ip-172-31-13-122 playbooks]# cat apache-install.yml
- name: Install Apache on Ubuntu and RHEL
  hosts: all
  become: yes

  tasks:
    - name: Install Apache on Ubuntu
      apt:
        name: apache2
        state: present
        update_cache: yes
        when: ansible_os_family == "Debian"

    - name: Start Apache on Ubuntu
      service:
        name: apache2
        state: started
        enabled: yes
        when: ansible_os_family == "Debian"

    - name: Install Apache on RHEL
      yum:
        name: httpd
        state: present
        when: ansible_os_family == "RedHat"

    - name: Start Apache on RHEL
      service:
        name: httpd
        state: started
        enabled: yes
        when: ansible_os_family == "RedHat"
```

Add below code:

---

- name: Install Apache on Ubuntu and RHEL

hosts: all

become: yes

tasks:

- name: Install Apache on Ubuntu

apt:

name: apache2

state: present

update\_cache: yes

when: ansible\_os\_family == "Debian"

- name: Start Apache on Ubuntu

service:

name: apache2

state: started

enabled: yes

when: ansible\_os\_family == "Debian"

- name: Install Apache on RHEL

yum:

name: httpd

state: present

when: ansible\_os\_family == "RedHat"

- name: Start Apache on RHEL

service:

name: httpd

state: started

enabled: yes

when: ansible\_os\_family == "RedHat"

→run code using “ansible-playbook -i /root/hosts/ apache.yml”

```
[root@ip-172-31-13-122 playbooks]# ansible-playbook -i /root/ansible/hosts apache-install.yml
PLAY [Install Apache on Ubuntu and RHEL] *****
TASK [Gathering Facts] *****
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]
[WARNING]: Platform linux on host worker-1 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [worker-1]
[WARNING]: Platform linux on host worker-2 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [worker-2]
TASK [Install Apache on Ubuntu] *****
skipping: [localhost]
skipping: [worker-1]
skipping: [worker-2]
TASK [Start Apache on Ubuntu] *****
skipping: [localhost]
skipping: [worker-1]
skipping: [worker-2]
TASK [Install Apache on RHEL] *****
changed: [localhost]
changed: [worker-1]
changed: [worker-2]
TASK [Start Apache on RHEL] *****
changed: [localhost]
changed: [worker-1]
changed: [worker-2]
PLAY RECAP *****
localhost                : ok=3    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
worker-1                  : ok=3    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
worker-2                  : ok=3    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
[root@ip-172-31-13-122 playbooks]#
```

### 3. Create ansible playbook using roles to configure LAMP stack

lamp/

└─ site.yml

└─ roles/

└─ apache/

│ └─ defaults/main.yml

│ └─ handlers/main.yml

│ └─ meta/main.yml

│ └─ tasks/main.yml

│ └─ templates/

│ └─ vars/main.yml

└─ mysql/

└─ php/

Go to ansible, cd /root/ansible



```
[root@ip-172-31-13-122 ~]# cd /root/ansible
[root@ip-172-31-13-122 ansible]# ls
hosts
[root@ip-172-31-13-122 ansible]#
```

## STEP 1: Decide where LAMP project should live

We'll keep it clean:

👉 LAMP project will be inside **/root/ansible**

cd /root/ansible

---

## ◆ STEP 2: Create LAMP project structure

mkdir lamp

cd lamp

mkdir roles

Now you are here:

/root/ansible/lamp/roles

---

## ◆ STEP 3: Run ansible-galaxy init (THIS IS THE KEY STEP)

👉 Run these commands **ONLY** here

👉 **Not** in playbook, not in vi

cd /root/ansible/lamp/roles

ansible-galaxy init apache

ansible-galaxy init mysql

ansible-galaxy init php

```
root@ip-172-31-13-122:~/ansible/lamp/roles
[root@ip-172-31-13-122 ansible]# mkdir lamp
[root@ip-172-31-13-122 ansible]# cd lamp
[root@ip-172-31-13-122 lamp]# mkdir roles
[root@ip-172-31-13-122 lamp]# cd roles/
[root@ip-172-31-13-122 roles]# ls
[root@ip-172-31-13-122 roles]# ansible-galaxy init apache
[root@ip-172-31-13-122 roles]# ansible-galaxy init mysql
[root@ip-172-31-13-122 roles]# ansible-galaxy init php
- Role apache was created successfully
- Role mysql was created successfully
- Role php was created successfully
[root@ip-172-31-13-122 roles]#
```

→ tree /root/ansible/lamp

```
[root@ip-172-31-13-122 roles]# tree /root/ansible/lamp
/root/ansible/lamp
├── roles
│   ├── apache
│   │   ├── README.md
│   │   ├── defaults
│   │   │   └── main.yml
│   │   ├── files
│   │   ├── handlers
│   │   │   └── main.yml
│   │   ├── meta
│   │   │   └── main.yml
│   │   ├── tasks
│   │   │   └── main.yml
│   │   ├── templates
│   │   ├── tests
│   │   │   ├── inventory
│   │   │   └── test.yml
│   │   └── vars
│   │       └── main.yml
│   ├── mysql
│   │   ├── README.md
│   │   ├── defaults
│   │   │   └── main.yml
│   │   ├── files
│   │   ├── handlers
│   │   │   └── main.yml
│   │   ├── meta
│   │   │   └── main.yml
│   │   ├── tasks
│   │   │   └── main.yml
│   │   ├── templates
│   │   ├── tests
│   │   │   ├── inventory
│   │   │   └── test.yml
│   │   └── vars
│   │       └── main.yml
│   └── php
│       ├── README.md
│       ├── defaults
│       │   └── main.yml
│       ├── files
│       ├── handlers
│       │   └── main.yml
│       ├── meta
│       │   └── main.yml
│       ├── tasks
│       │   └── main.yml
│       ├── templates
│       ├── tests
│       │   ├── inventory
│       │   └── test.yml
│       └── vars
│           └── main.yml
28 directories, 24 files
```

## Steps Followed to Configure LAMP Stack Using Ansible Galaxy Roles

---

### ◆ Step 1: Set Up Ansible Environment

- Installed Ansible on the control node.
- Created an inventory file with:
  - **Controller** (localhost)
  - **Worker nodes** (Amazon Linux EC2 instances)
- Configured SSH key-based authentication for passwordless access.

---

### ◆ Step 2: Verified Inventory and Connectivity

- Used the inventory file located at:
  - /root/ansible/hosts

- Verified connectivity using:
  - `ansible -i /root/ansible/hosts workers -m ping`
  - Confirmed Ansible could communicate with all worker nodes.
- 

#### ◆ Step 3: Created a New Project for LAMP Stack

- Created a dedicated project directory:
  - `/root/ansible/lamp`
  - Followed Ansible best practices by separating this project from other playbooks.
- 

#### ◆ Step 4: Created Roles Using Ansible Galaxy

- Navigated to the roles directory:
  - `cd /root/ansible/lamp`
  - `mkdir roles`
  - `cd roles`
  - Used **Ansible Galaxy** to generate standard role structures:
  - `ansible-galaxy init apache`
  - `ansible-galaxy init mysql`
  - `ansible-galaxy init php`
- 

#### ◆ Step 5: Created the Main Playbook (site.yml)

- Created `site.yml` to orchestrate the roles.
- Ensured the playbook only referenced roles (no tasks inside it).

- hosts: workers

become: yes

roles:

- apache

- mysql

- php

---

#### ◆ Step 6: Configured Apache Role

- Added OS-based logic inside:
- roles/apache/tasks/main.yml
- Installed Apache using:
  - apache2 on Ubuntu
  - httpd on RHEL/Amazon Linux
- Started and enabled the Apache service.

---

#### ◆ Step 7: Configured MySQL (MariaDB) Role

- Added tasks inside:
- roles/mysql/tasks/main.yml
- Implemented OS-aware installation:
  - Ubuntu → mariadb-server
  - RHEL → mariadb-server
  - Amazon Linux → mariadb105-server
- Handled the Amazon Linux edge case by excluding it from RHEL tasks.
- Started and enabled the MariaDB service.

---

#### ◆ Step 8: Configured PHP Role

- Added PHP installation logic inside:
- roles/php/tasks/main.yml
- Installed PHP packages based on OS:
  - Ubuntu → php, php-mysql
  - RHEL/Amazon Linux → php, php-mysqlnd

---

#### ◆ Step 9: Executed the LAMP Playbook

- Ran the playbook using the correct inventory:

- `ansible-playbook -i /root/ansible/hosts site.yml`
- Observed task execution based on OS conditions.

---

### ◆ Step 10: Verified Successful Deployment

- Verified Apache installation via browser or curl.
- Checked MariaDB service status.
- Verified PHP installation using:
  - `php -v`
- Confirmed there were **no failures** in the play recap.

---

### ✓ Final Outcome

- Successfully deployed a **LAMP stack** on **Amazon Linux** worker nodes.
- Followed **Ansible Galaxy standards**.
- Used **roles for modular and reusable configuration**.
- Implemented **OS-specific logic** to handle package differences.
- Achieved a clean, production-ready automation setup.

→ `cd /root/ansible/lamp`

→ `vi site.yml`



```
root@ip-172-31-13-122:~/ansible/lamp
--
- name: Configure LAMP stack using Ansible Galaxy roles
  hosts: workers
  become: yes

  roles:
    - apache
    - mysql
    - php
```

→ `vi roles/apache/tasks/main.yml`



```
root@ip-172-31-13-122:~/ansible/lamp
--
- name: Install Apache on Ubuntu
  apt:
    name: apache2
    state: present
    update_cache: yes
    when: ansible_os_family == "Debian"

- name: Install Apache on RHEL
  yum:
    name: httpd
    state: present
    when: ansible_os_family == "Redhat"

- name: Start Apache
  service:
    name: "[[ 'apache2' if ansible_os_family == 'Debian' else 'httpd' ]]"
    state: started
    enabled: yes
```

→ `vi roles/mysql/tasks/main.yml`

```
root@ip-172-31-13-122:~/ansible/lamp
- name: Install MariaDB
  package:
    name: mariadb-server
    state: present

- name: Start MariaDB
  service:
    name: mariadb
    state: started
    enabled: yes
```

→ vi roles/php/tasks/main.yml

```
root@ip-172-31-13-122:~/ansible/lamp
# Install PHP on Ubuntu
- name: Install PHP on Ubuntu
  apt:
    name:
      - php
      - php-mysql
    state: present
    update_cache: yes
    when: ansible_os_family == "Debian"

# Install PHP on RHEL & Amazon Linux
- name: Install PHP on RHEL and Amazon Linux
  yum:
    name:
      - php
      - php-mysqlnd
    state: present
    when: ansible_os_family == "Redhat"
```

→ ansible -i /root/ansible/hosts workers -m ping

→ ansible -i /root/ansible/hosts workers -m ping

```
root@ip-172-31-13-122:~/ansible/lamp# vi roles/php/tasks/main.yml
[root@ip-172-31-13-122:~]# ansible -i /root/ansible/hosts workers -m ping
[WARNING]: Platform linux on host worker-1 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
worker-1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Platform linux on host worker-2 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
worker-2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[root@ip-172-31-13-122:~]#
```

→ ansible-playbook -i /root/ansible/hosts site.yml

```
PLAY [Configure LAMP stack using Ansible galaxy roles] *****
TASK [Gathering Facts] *****
[WARNING]: Platform linux on host worker-1 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [worker-1]
[WARNING]: Platform linux on host worker-2 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [worker-2]

TASK [apache : Install Apache on Ubuntu] *****
skipping: [worker-1]
skipping: [worker-2]

TASK [apache : Install Apache on RHEL] *****
ok: [worker-1]
ok: [worker-2]

TASK [apache : Start Apache] *****
ok: [worker-1]
ok: [worker-2]

TASK [mysql : Install MariaDB on Ubuntu] *****
skipping: [worker-1]
skipping: [worker-2]

TASK [mysql : Install MariaDB on RHEL] *****
skipping: [worker-1]
skipping: [worker-2]

TASK [mysql : Install MariaDB on Amazon Linux] *****
ok: [worker-2]
ok: [worker-1]

TASK [mysql : Start MariaDB] *****
ok: [worker-1]
ok: [worker-2]

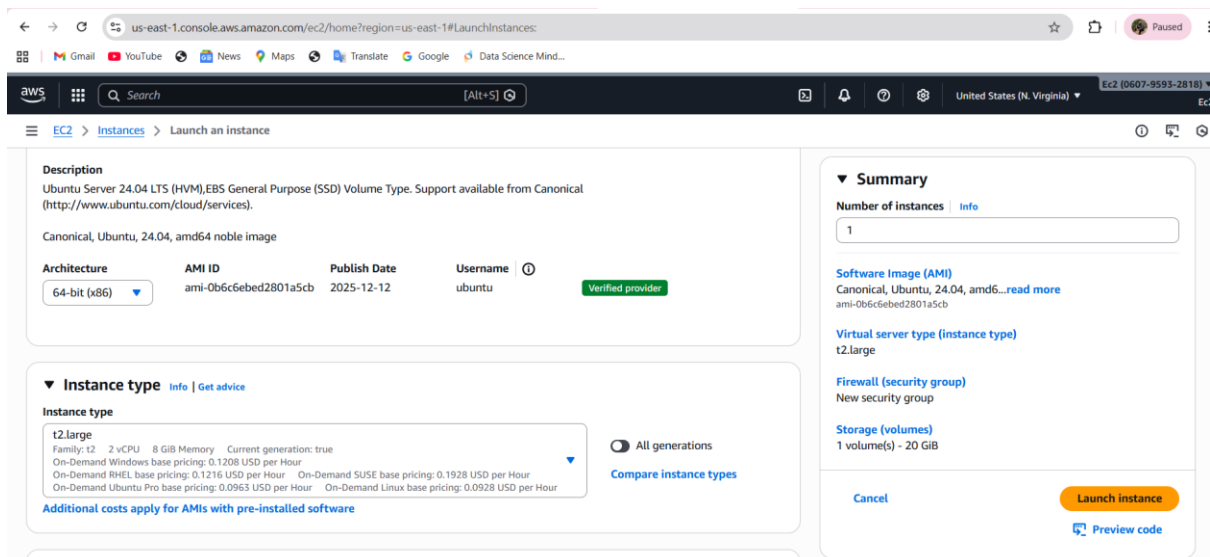
TASK [php : Install PHP on Ubuntu] *****
skipping: [worker-1]
skipping: [worker-2]

TASK [php : Install PHP on RHEL and Amazon Linux] *****
changed: [worker-1]
changed: [worker-2]

PLAY RECAP *****
worker-1 : ok=6 changed=1 unreachable=0 failed=0 skipped=4 rescued=0 ignored=0
worker-2 : ok=6 changed=1 unreachable=0 failed=0 skipped=4 rescued=0 ignored=0
[root@ip-172-31-13-122:~]#
```

## 4. Setup ansible AWX and explore the options.

→ created one instance with 20gb volume and t2.large instance type.

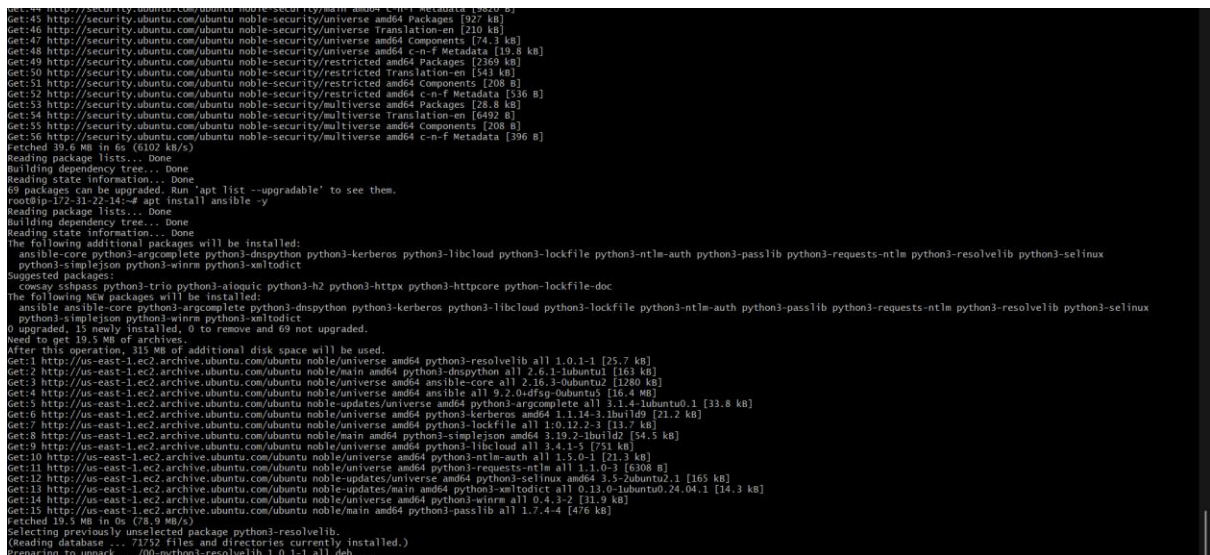


→ Connect with ssh

→ install ansible

→ Install docker

→ apt install docker.io



→ sudo systemctl enable docker

→ sudo systemctl start docker

→ sudo systemctl status docker

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-22-14:~# sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-02 12:47:59 UTC; 1min 14s ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2916 (dockerd)
      Tasks: 9
     Memory: 21.6M (peak: 21.8M)
        CPU: 339ms
     CGroup: /system.slice/docker.service
             └─2916 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.971782613Z" level=info msg="Loading containers: start."
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.293857759Z" level=info msg="Loading containers: done."
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.310634531Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1-24.04.1" containerd-snapshotter=false storage-driver=overlay2 version=
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.310714935Z" level=info msg="Initializing buildkit"
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.31750363Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.31773994Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.337295418Z" level=info msg="Completed buildkit initialization"
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.346891122Z" level=info msg="Daemon has completed initialization"
Feb 02 12:47:59 ip-172-31-22-14 dockerd[2916]: time="2026-02-02T12:47:59.346941674Z" level=info msg="API listen on /run/docker.sock"
Feb 02 12:47:59 ip-172-31-22-14 systemd[1]: Started docker.service - Docker Application Container Engine.

root@ip-172-31-22-14:~# |
```

→ apt install docker-compose

```
After this operation, 1589 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-websocket all 1.7.0-1 [38.1 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-docker all 5.0.3-1ubuntu1.1 [89.1 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-dockerty all 0.4.1-5 [11.4 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-dcopt all 0.6.2-6 [26.1 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-dotenv all 1.0.1-1 [22.3 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-texttable all 1.6.7-1 [11.0 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-compose all 1.29.2-6ubuntu1 [84.6 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 docker-compose all 1.29.2-6ubuntu1 [14.0 kB]
Fetched 297 kB in 0s (6670 kB/s)
Selecting previously unselected package python3-websocket.
(Reading database ... 95182 files and directories currently installed.)
Preparing to unpack .../0-python3-websocket_1.7.0-1_all.deb ...
Unpacking python3-websocket (1.7.0-1) ...
Selecting previously unselected package python3-docker.
Preparing to unpack .../1-python3-docker_5.0.3-1ubuntu1.1_all.deb ...
Unpacking python3-docker (5.0.3-1ubuntu1.1) ...
Selecting previously unselected package python3-dockerty.
Preparing to unpack .../2-python3-dockerty_0.4.1-5_all.deb ...
Unpacking python3-dockerty (0.4.1-5) ...
Selecting previously unselected package python3-dcopt.
Preparing to unpack .../3-python3-dcopt_0.6.2-6_all.deb ...
Unpacking python3-dcopt (0.6.2-6) ...
Selecting previously unselected package python3-dotenv.
Preparing to unpack .../4-python3-dotenv_1.0.1-1_all.deb ...
Unpacking python3-dotenv (1.0.1-1) ...
Selecting previously unselected package python3-texttable.
Preparing to unpack .../5-python3-texttable_1.6.7-1_all.deb ...
Unpacking python3-texttable (1.6.7-1) ...
Selecting previously unselected package python3-compose.
Preparing to unpack .../6-python3-compose_1.29.2-6ubuntu1_all.deb ...
Unpacking python3-compose (1.29.2-6ubuntu1) ...
Selecting previously unselected package docker-compose.
Preparing to unpack .../7-docker-compose_1.29.2-6ubuntu1_all.deb ...
Unpacking docker-compose (1.29.2-6ubuntu1) ...
Setting up python3-dotenv (1.0.1-1) ...
Setting up python3-texttable (1.6.7-1) ...
Setting up python3-dcopt (0.6.2-6) ...
Setting up python3-websocket (1.7.0-1) ...
Setting up python3-dockerty (0.4.1-5) ...
Setting up python3-docker (5.0.3-1ubuntu1.1) ...
Setting up python3-compose (1.29.2-6ubuntu1) ...
Setting up docker-compose (1.29.2-6ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

→ cd ~

→git clone -b 17.1.0 https://github.com/ansible/awx.git

→cd awx

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-22-14:~# cd ~
root@ip-172-31-22-14:~# git clone -b 17.1.0 https://github.com/ansible/awx.git
Cloning into 'awx'...
remote: Enumerating objects: 329060, done.
remote: Counting objects: 100% (135/135), done.
remote: Compressing objects: 100% (67/67), done.
remote: Total 329060 (delta 92), reused 68 (delta 68), pack-reused 328925 (from 2)
Receiving objects: 100% (329060/329060), 345.38 MiB | 40.34 MiB/s, done.
Resolving deltas: 100% (252344/252344), done.
Note: switching to 'c1ab815c80cac96508d9779d92bc1280d0347627'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

root@ip-172-31-22-14:~/awx# |
```

→"vi ~/awx/installer/inventory"

→add admin\_password=Admin@123

secret\_key=Admin@123

pg\_database=awx

pg\_password=Admin@123

awx\_alternate\_dns\_servers="8.8.8.8,8.8.4.4"



```
postgres_data_dir="/var/lib/awx/pgdocker"
docker_compose_dir="/var/lib/awx/awxcompose"
project_data_dir="/var/lib/awx/projects"
```

```
root@ip-172-31-22-14: ~/awx/installer
# Review the trademark guidelines at https://github.com/ansible/awx-logos/blob/master/TRADEMARKS.md
# awx_official=False

# Proxy
#http_proxy=http://proxy:3128
#https_proxy=http://proxy:3128
#no_proxy=mycorp.org

# Container networking configuration
# Set the awx_task and awx_web containers' search domain(s)
#awx_container_search_domains=example.com,ansible.com
# Alternate DNS servers
#awx_alternate_dns_servers="10.1.2.3,10.2.3.4"

# AWX project data folder. If you need access to the location where AWX stores the projects
# it manages from the docker host, you can set this to turn it into a volume for the container.
#project_data_dir=/var/lib/awx/projects

# AWX custom virtual environment folder, only usable for local install.
#custom_venv_dir=/opt/my-venv/

# CA Trust directory. If you need to provide custom CA certificates, supplying
# this variable causes this directory on the host to be bind mounted over
# /etc/pki/ca-trust in the awx_task and awx_web containers
# If you are deploying on openshift or kubernetes, set the variable to /etc/pki/ca-trust instead,
# as the awx_web and awx_task containers will not run the update-ca-trust command.
#ca_trust_dir=/etc/pki/ca-trust/source/anchors

# Include /etc/nginx/awx_extra.conf
# Note the use of glob pattern for nginx
# which makes include "optional" - i.e. not fail
# If file is absent
#extra_nginx_include="/etc/nginx/awx_extra[.]conf"

# Docker compose explicit subnet. Set to avoid overlapping your existing LAN networks.
#docker_compose_subnet="172.17.0.1/16"
#
# Allow for different docker logging drivers
# By Default, the logger will be json-file, however you can override
# that by uncommenting the docker_logger below.
# Be aware that Journald may rate limit your log messages if you choose it.
# See: https://docs.docker.com/config/containers/logging/configure/
# docker_logger=journald
#
# Add extra hosts to docker compose file. This might be necessary to
# sneak in servernames. For example for DMZ self-signed CA certificates.
# Equivalent to using the --add-host parameter with "docker run".
#admin_password=Admin@123
#secret_key=Admin@123
#pg_database=awx
#pg_password=Admin@123
#awx_alternate_dns_servers="8.8.8.8,8.8.4.4"
#postgres_data_dir="/var/lib/awx/pgdocker"
#docker_compose_dir="/var/lib/awx/awxcompose"
#project_data_dir="/var/lib/awx/projects"
#docker_compose_extra_hosts="--otherserver.local:192.168.0.1,ldap-server.local:192.168.0.2"
-- INSERT --
```

→ ansible-playbook -i ~/awx/installer/inventory ~/awx/installer/install.yml

```
root@ip-172-31-22-14: ~/awx/installer
TASK [local_docker : Set up new postgres paths pre-upgrade] *****
skipping: [localhost]

TASK [local_docker : Stop AWX before upgrading postgres] *****
skipping: [localhost]

TASK [local_docker : Upgrade Postgres] *****
skipping: [localhost]

TASK [local_docker : Copy old pg_hba.conf] *****
skipping: [localhost]

TASK [local_docker : Remove old data directory] *****
skipping: [localhost]

TASK [local_docker : Export Docker awx image if it isnt local and there isnt a registry defined] *****
skipping: [localhost]

TASK [local_docker : Set docker base path] *****
skipping: [localhost]

TASK [local_docker : Ensure directory exists] *****
skipping: [localhost]

TASK [local_docker : Copy awx image to docker execution] *****
skipping: [localhost]

TASK [local_docker : Load awx image] *****
skipping: [localhost]

TASK [local_docker : Set full image path for local install] *****
skipping: [localhost]

TASK [local_docker : Set DockerHub Image Paths] *****
skipping: [localhost]

TASK [local_docker : Create /var/lib/awx/awxcompose directory] *****
changed: [localhost]

TASK [local_docker : Create Redis socket directory] *****
changed: [localhost]

TASK [local_docker : Create Docker Compose Configuration] *****
changed: [localhost] => (item={'file': 'environment.sh', 'mode': '0600'})
changed: [localhost] => (item={'file': 'credentials.py', 'mode': '0600'})
changed: [localhost] => (item={'file': 'docker-compose.yml', 'mode': '0600'})
changed: [localhost] => (item={'file': 'nginx.conf', 'mode': '0600'})
changed: [localhost] => (item={'file': 'redis.conf', 'mode': '0664'})

TASK [local_docker : Render SECRET_KEY file] *****
changed: [localhost]

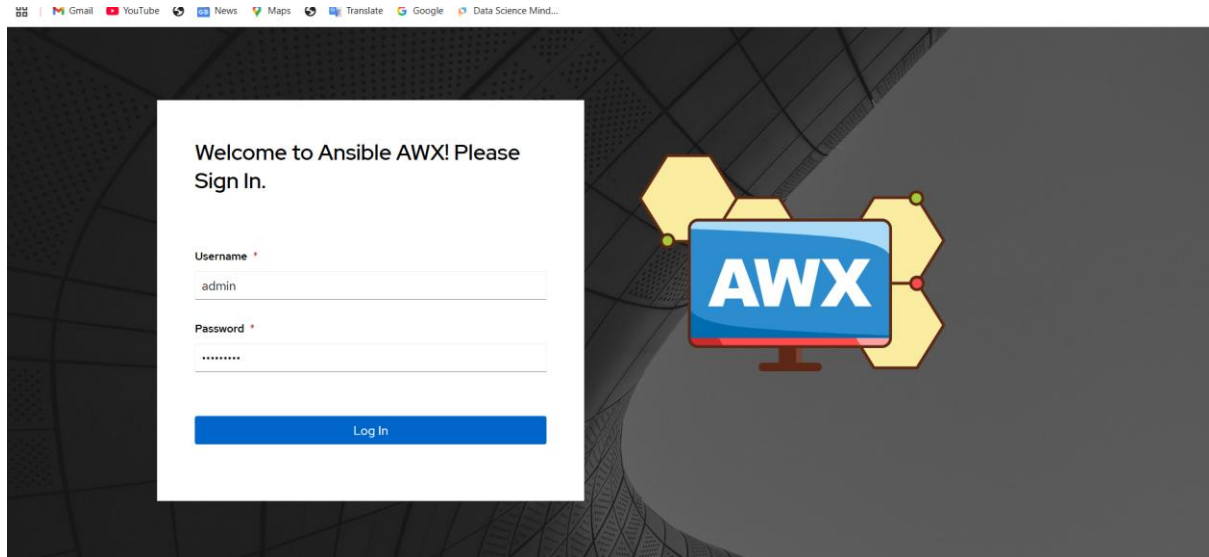
TASK [local_docker : Remove awx containers before migrating postgres so that the old postgres container does not get used] *****
ok: [localhost]

TASK [local_docker : Run migrations in task container] *****
```

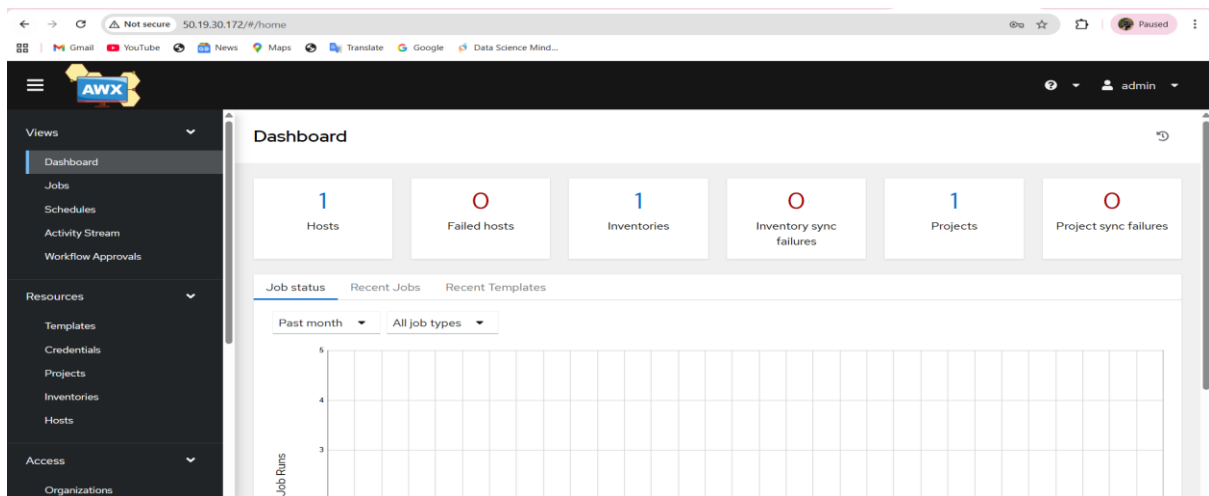
→login into the UI AWX

→username: admin

→password: Admin@123 (by default)



→Logged in successfully



→worker-1 added in the customized inventory.

